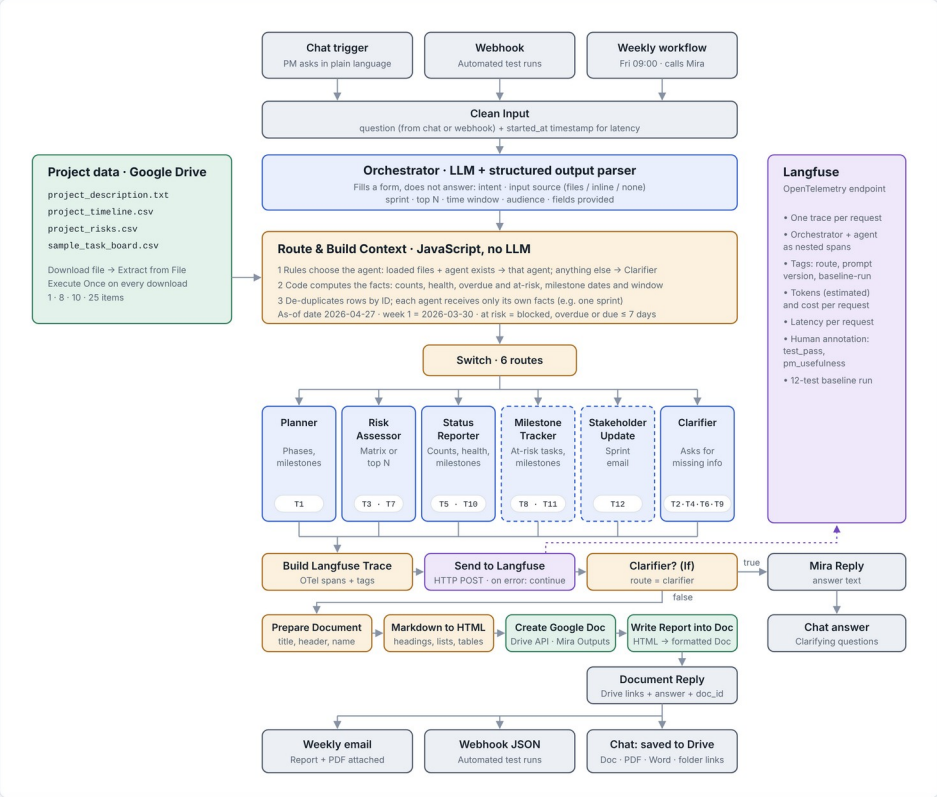
Q2 · Build Mira: Write-up

Mira · AI Project Intelligence Assistant for Nexora · Interview Kickstart, Applied Agentic AI for PMs/TPMs · Ashutosh Lonkar · November 2026

Mira is a multi-agent n8n workflow that answers Nexora PMs' questions about the ABCDE Ltd. AI Adoption Project: project plans, risk assessments, weekly status reports, milestone alerts and stakeholder emails. Every answer is grounded in the four project files, vague requests get clarifying questions instead of invented content, and every request is traced in Langfuse. **Result: 12 of 12 baseline tests pass, in both the v1 baseline and the v2 regression run, at about $0.0005 per request.**

1. Design and architecture



Design principle: code computes, the LLM writes. LLMs miscount rows and get date arithmetic wrong. Every count, date comparison and filter therefore runs in a JavaScript Code node, and each agent receives only the facts it needs, with an instruction to copy them exactly. This is what makes the answers checkable against the files.

Step	Node(s)	What happens
Input	Chat Trigger, Webhook, Execute Workflow Trigger → Clean Input	One entry point for the PM chat, automated test runs and the weekly workflow; text is normalized and a start time recorded
Classify	Orchestrator (LLM + Structured Output Parser, temp 0)	Fills a form: intent, input source (loaded files, quoted text, none), sprint, top N, time window, audience. It does not answer
Ground	4 × Google Drive Download → Extract from File (Execute Once)	Project description, timeline, task board and risk register are read on every request
Compute	Route & Build Context (Code)	Rules choose the agent; code computes counts, Red/Amber/Green health, overdue and at-risk tasks, dated timeline, upcoming milestones; rows de-duplicated by ID
Answer	Switch → 6 Basic LLM Chains (gpt-4o-mini)	One specialist agent writes the answer from its own facts

Step	Node(s)	What happens
Observe	Build Langfuse Trace → HTTP Request (OpenTelemetry)	One trace per request with orchestrator and agent spans, tags, tokens, cost, latency
Deliver	Markdown to HTML → Drive API → Document Reply	Clarifier questions answered in chat; all other answers saved as a Google Doc with PDF, Word and folder links

Orchestration pattern: Router. Requests vary, and each needs different data and a different prompt: a plan needs the description and timeline, a status report needs the task board. A sequential pipeline would run every agent on every request, cost about six times more, and let agents answer questions nobody asked. A hierarchical pattern adds a planning loop that single-intent requests do not need. The router's weakness is that it is a single point of failure, so its decision is recorded on every trace and checked in testing (12 of 12 routed correctly). The known limit is that a two-intent request ("plan and top risks") gets one answer.

Hybrid routing. The LLM only classifies; deterministic rules in code pick the agent. A request about the loaded files goes to its specialist; a request with no data, quoted vague text or another project goes to the Clarifier. This keeps routing explainable and repeatable.

Tool choice: n8n over Langflow. Mira's hard parts are integration and determinism, not retrieval. n8n has native Google Drive, Gmail, chat, webhook and schedule triggers, sub-workflow calls and JavaScript Code nodes in one canvas; Langflow is stronger for LLM-centric prototyping and vector retrieval but would need custom components for Drive, email and scheduling. The project files are small (25 tasks, 10 risks), so they fit in context without a vector store: grounding is done by passing computed facts, not by retrieval. One main workflow keeps every step in a single execution and trace; the weekly report is a separate scheduled workflow that calls Mira as a sub-workflow, so both share one implementation.

2. Implementation

Core and extended capabilities

Capability	Agent (temperature)	What it produces	Tests
Project Plan Generation (core)	Planner (0.2)	8 phases with weeks, activities, 14 milestones, goals copied exactly; start date, budget and owners raised as open questions, not invented	T1
Risk Assessment (core)	Risk Assessor (0.2)	Full matrix of all 10 risks (R01–R10) or top N, with category, impact, mitigation and reasoning; caveat that the register has no severity scores	T3, T7
Weekly Status Report (core)	Status Reporter (0.1)	Health with reason, counts by status, tasks by status, needs attention, upcoming milestones; for one sprint or the whole project	T5, T10
Milestone Alert System (extended)	Milestone Tracker (0.2)	Blocked and at-risk tasks with owner, due date, reason and a suggested action; milestones in a time window	T8, T11
Stakeholder Update Generator (extended)	Stakeholder Update (0.4)	Sprint update email for leadership, engineering or the client; only that sprint's tasks reach the agent	T12
Clarification (enhancement)	Clarifier (0)	Says the input is insufficient and asks specific questions; never generates a plan, risk list or status	T2, T4, T6, T9

Prompt engineering. Each system prompt names its facts block, requires a fixed Markdown structure, forbids inventing tasks, risks, dates or people, and says what to do when data is missing. Versions are tracked (v1 baseline, v2 adds upcoming milestones) and tagged on every trace; all seven prompts are in docs/prompts.md.

Enhancements beyond the brief. Reports saved to Google Drive with one-click PDF and Word downloads; a scheduled Friday 09:00 workflow that emails the current-sprint report with the PDF attached; a de-duplication guard; and a human evaluation queue in Langfuse.

Observability

Langfuse is connected through its OpenTelemetry endpoint (the legacy ingestion API rejected new projects). Each request produces one trace with nested orchestrator and agent spans, input and output, route, sprint, prompt version, estimated tokens, cost and latency. Tags (baseline-run, regression-v2, route) let runs be filtered and compared. An annotation queue holds two human scores per trace: test_pass (Pass/Fail) and pm_usefulness (1–5). The dashboard shows 26 traces at $0.0168 in total.

Baseline testing

Metric	v1 baseline (29 Oct)	v2 regression (5–6 Nov)
Tests passed (brief's expected-output criteria)	12 / 12	12 / 12
Requests routed to the intended agent	12 / 12	12 / 12
Correct refusals on vague input (T2, T4, T6, T9)	4 / 4	4 / 4
Counts and task IDs matching the source files	100%	100%
Status reports with upcoming milestones (T5, T10)	0 / 2	2 / 2
Tests with a quality defect remaining	10	6
Average PM usefulness (human score, 1–5)	4.5	Pass/fail scored
Average latency per request	10.6 s	10.2 s

Per-test inputs, answers and defects are in docs/baseline_results.md and docs/Mira_Test_Log_v2.xlsx.

3. Cost and evaluation

Cost measure	Value	Basis
Cost per project plan (T1)	$0.00076	≈1,940 input + 780 output tokens on gpt-4o-mini
Average cost per request	$0.00046 (v1) · $0.00050 (v2)	All agents; v2 slightly higher because status facts now include milestones
Most expensive request type	$0.00081 (full status report, T10)	Largest facts block; still under a tenth of a cent
Projected monthly cost	$0.83	10 PMs × 8 requests/day × 21 days = 1,680 requests
Cost of not using Mira	10–20 PM-hours per week	1–2 hours per PM per week on manual status reports

Tokens are estimated from characters ÷ 4: the order of magnitude is reliable, exact cents are not. Google Drive and Gmail calls are free at this volume.

Evaluation strategy. Each test is judged against the brief's expected output on three levels: (1) **grounding**, checked exactly by comparing every count, task ID, risk ID and date with the files; (2) **behaviour**, i.e. correct routing and refusing vague input instead of inventing; (3) **usefulness**, a PM's 1–5 score in the Langfuse annotation queue. Latency and cost come from the traces. Every prompt change is re-run on all 12 inputs as a regression test and compared by tag.

Observations

- **Grounding held in every run.** Re-running a test changed the wording but never the facts, because the facts are computed in code.
- **The brief's summary and the data disagree:** the brief says 3 tasks are Done, the task board has 5. Mira reports the data.
- **Volume bug caught by counts.** A Download node without Execute Once re-read a file once per upstream item and inflated counts 80 times; fixed with Execute Once and de-duplication by ID.
- **Evaluation found a core gap.** v1 status reports had no upcoming milestones; v2 added them and the regression run confirmed it with no other test affected.
- **Remaining defects are wording, not facts:** a T10 counts-only question got a full report, T12 used unsupported filler ("significant progress"), and T3/T7 ranked the same three risks in a different order. These are planned for v3 (Q4).